

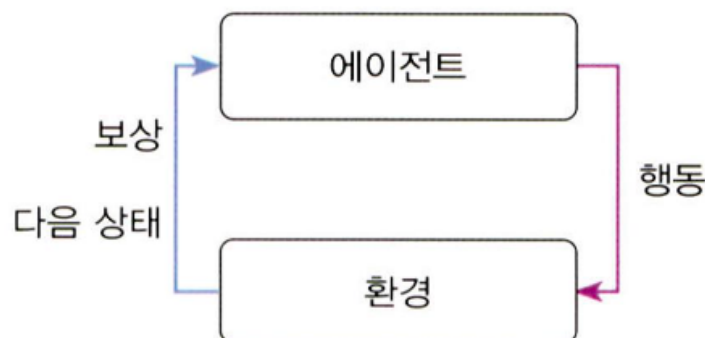
3. 강화학습 개념

1장. 강화학습 개요

1.1 강화학습이란

: 에이전트가 환경과 상호작용하며 보상을 최대화하는 정책을 학습하는 방법

- 정답 label이 직접 주어지지 않음
- 행동의 결과로 받는 **보상**을 통해 학습
- 시행착오 기반 학습
- 최종 목표: **최적 정책 optimal policy** 찾기



1.2 강화 개념의 출발: 스키너 실험

: 행동 뒤에 보상이 주어지면 그 행동을 더 자주 하게 되는 현상

- 쥐가 우연히 지렛대를 누름 → 먹이 보상 획득 → 반복하면서 지렛대와 먹이의 관계 학습 → 결국 보상을 주는 행동을 더 자주 선택
- 원리를 이해한 것이 아니라, **행동** → **결과**의 연결을 학습한 것

1.3 우리 주변의 강화

: 사람도 직접 해 보고 결과를 통해 배우는 경우가 많음

- 걸음마: 넘어지며 균형 학습

- 자전거: 시행착오로 조작 학습
- 알파고: 반복 대국으로 이기는 전략 학습

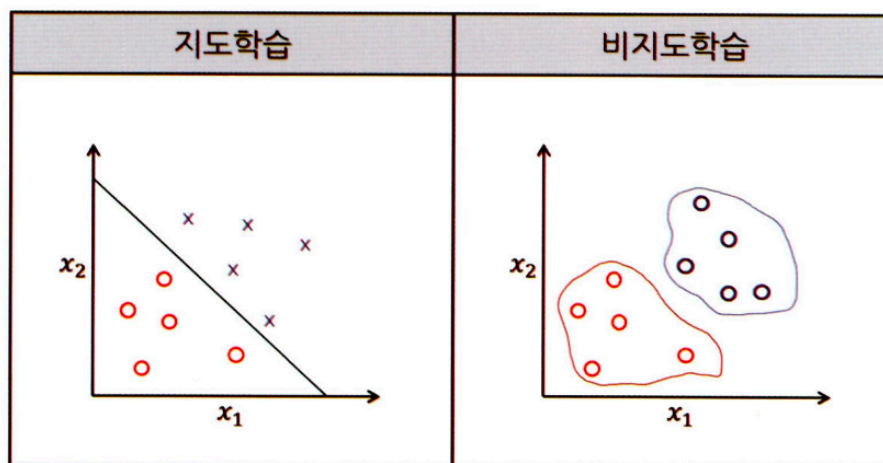
강화학습의 장점

- 환경의 정확한 사전지식이 없어도 학습 가능
- 직접 상호작용하며 기능 습득 가능

1.4 머신러닝과 강화학습

: 머신러닝은 지도학습, 비지도학습, 강화학습으로 구분

- 지도학습: 정답이 있는 데이터로 학습
- 비지도학습: 정답 없이 데이터 구조 학습
- 강화학습: 환경과 상호작용하며 보상으로 학습



강화학습의 차이점

- 지도학습처럼 직접적인 정답 없음
- 비지도학습처럼 데이터 구조만 찾는 것도 아님
- 보상 기반 의사결정 학습이라는 점이 핵심

1.5 강화학습 문제

: 강화학습은 순차적으로 행동을 결정해야 하는 문제를 다룸

- 현재 행동이 다음 상태에 영향

- 다음 상태가 이후 행동과 보상에 영향
- 한 번의 선택으로 끝나는 문제가 아님

예시

- 게임, 로봇 제어, 경로 탐색, 장기적 보상을 고려해야 하는 의사결정 문제

1.6 강화학습 문제를 이루는 기본 요소

: 강화학습 문제는 수학적으로 정의되어야 하며, 그 기본 틀이 MDP로 이어짐

- **상태 State**: 에이전트가 관찰하는 현재 상황
- **행동 Action**: 각 상태에서 선택 가능한 행동
- **보상 Reward**: 행동 결과의 좋고 나쁨을 알려 주는 값
- **정책 Policy**: 각 상태에서 어떤 행동을 할지 정하는 기준

상태에서 중요한 점

- 행동 결정을 하기에 충분한 정보가 포함되어야 함
- 위치만이 아니라 속도 같은 정보도 필요할 수 있음

1.7 큰 상태공간과 강화학습의 확장

: 상태공간이 커질수록 강화학습은 더 어려워지고, 그래서 신경망과의 결합이 중요해짐

- 바둑처럼 경우의 수가 매우 큰 문제 존재
- 모든 상태를 표로 저장하거나 직접 계산하기 어려움
- 그래서 함수 근사, 특히 신경망과의 결합이 중요

의미

- 강화학습은 게임을 넘어서, 실제 로봇 문제 같은 현실 환경으로 확장되는 중

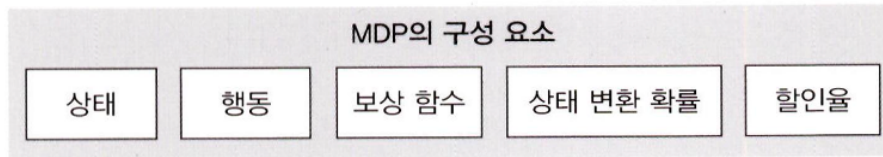
2장. MDP와 벨만 방정식

2.1 MDP: 강화학습 문제를 수학적으로 표현하는 틀

: **Markov Decision Process**

“에이전트가 어떤 상황에서 어떤 선택을 하고, 그 결과가 다음 상황과 보상으로 어떻게 이어지는가”

MDP의 기본 구성 요소



- **상태 (S):** 에이전트가 관찰하는 현재 상황의 집합
- **행동 (A):** 각 상태에서 선택할 수 있는 행동의 집합
- **보상 함수:** 특정 상태에서 특정 행동을 했을 때 받는 즉각적인 보상의 기댓값
- **상태변환확률:** 상태 (s)에서 행동 (a)를 했을 때 다음 상태 (s')로 이동할 확률
- **할인율:** 미래 보상을 현재 가치로 얼마나 반영할지 결정하는 비율

2.2 MDP를 이루는 핵심 개념

: 상태, 행동, 보상, 할인율, 정책

$$S = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), (x_4, y_4), (x_5, y_5)\}$$

수식 2.1 상태의 집합

(1, 1)	(2, 1)	(3, 1)	(4, 1)	(5, 1)
(1, 2)	(2, 2)	(3, 2)	(4, 2)	(5, 2)
(1, 3)	(2, 3)	(3, 3)	(4, 3)	(5, 3)
(1, 4)	(2, 4)	(3, 4)	(4, 4)	(5, 4)
(1, 5)	(2, 5)	(3, 5)	(4, 5)	(5, 5)

그림 2.2 그리드월드에서 상태는 좌표를 의미한다

격자 환경이라면 각 칸 하나하나가 상태가 되고, 에이전트는 각 상태에서 위, 아래, 왼쪽, 오른쪽 같은 행동을 선택할 수 있음

→ 에이전트가 어느 칸에 있고 어디로 움직일 수 있는지가 명확해짐

2.3 상태, 행동, 보상함수, 할인율

: MDP의 각 요소는 강화학습 문제를 구체적으로 정의하는 역할

상태 State

- 에이전트가 관찰하는 현재 상태
- 행동 결정에 충분한 정보 포함 필요
- 예: 그리드월드에서는 현재 좌표

행동 Action

- 상태에서 선택 가능한 행동의 집합
- 예: `up`, `down`, `left`, `right`

보상함수 Reward Function

- 상태 (s)에서 행동 (a)를 했을 때 받는 보상의 기댓값
- 강화학습의 목표는 누적 보상 최대화

할인율 Discount Factor

- 미래 보상을 현재 가치로 환산하는 비율
- ($\gamma = 0$): 현재 보상만 고려
- ($\gamma = 1$): 미래 보상도 현재와 동일하게 고려
- ($\gamma \in (0,1)$): 미래일수록 가치 감소

2.4 정책, 반환값, 가치함수, 큐함수

: 정책은 행동 선택 기준, 가치함수와 큐함수는 상태와 행동의 가치를 수치화한 것

정책 Policy

- 각 상태에서 어떤 행동을 선택할지 정하는 함수
- 강화학습의 목표는 최적 정책 π^* 찾기

반환값 Return

- 현재 이후 받을 할인된 보상의 합
- 즉시 보상만이 아니라 미래 보상까지 함께 고려

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \gamma^4 R_{t+5} \dots$$

수식 2.18 할인율을 적용한 보상들의 합

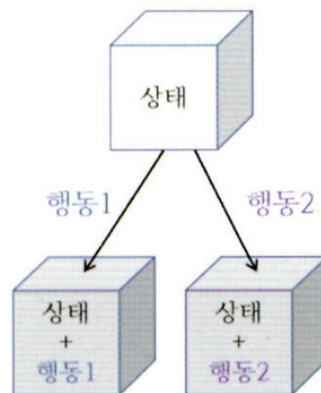
가치함수 Value Function

- 정책 (π)를 따를 때 상태 (s)의 기대 반환값

$$v(s) = E[G_t | S_t = s]$$

- “이 상태가 얼마나 좋은가”를 수치화한 값

큐함수 Q-Function



- 상태 (s)에서 행동 (a)를 했을 때의 기대 반환값
- 행동별 가치를 직접 비교 가능
- 실제 강화학습에서 더 자주 사용됨

$$q_{\pi}(s, a) = E_{\pi}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

2.5 벨만 기대 방정식

: 현재 상태의 가치는 즉시 보상과 다음 상태 가치의 할인된 합으로 표현 가능

가치함수의 벨만 기대 방정식

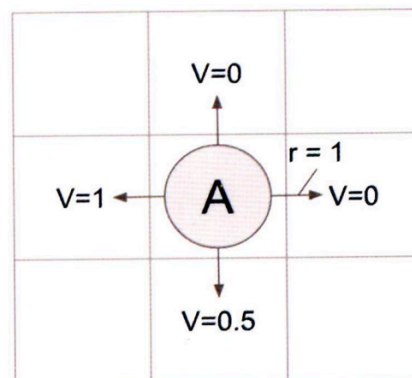
$$v_{\pi}(s) = E_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s]$$

- 현재 가치 = 즉시 보상 + 미래 가치
- 강화학습의 재귀적 구조를 보여 주는 식

전개된 형태

$$v_{\pi}(s) = \sum_{a \in A} \pi(a \mid s) \left(r_{(s,a)} + \gamma \sum_{s' \in S} P_{ss'}^a v_{\pi}(s') \right)$$

- 정책에 따른 행동 선택
- 상태전이확률에 따른 다음 상태 이동
- 그 결과의 기댓값 계산



2.6 벨만 최적 방정식과 최적 정책

: 최적 가치함수와 최적 큐함수는 가능한 행동 중 최대값을 취하는 형태로 표현

최적 가치함수, 최적 큐함수

$$v_*(s) = \max_{\pi} [v_{\pi}(s)]$$

수식 2.36 최적의 가치함수

$$q_*(s,a) = \max_{\pi} [q_{\pi}(s,a)]$$

수식 2.37 최적의 큐함수

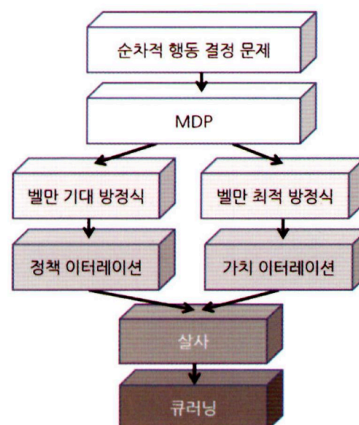
최적 정책

$$\pi_*(s,a) = \begin{cases} 1 & \text{if } a = \operatorname{argmax}_{a \in A} q_*(s,a) \\ 0 & \text{otherwise} \end{cases}$$

- 최적 큐함수를 알면 최적 정책을 바로 구할 수 있음
- 그래서 많은 강화학습 알고리즘이 큐함수 추정 중심으로 전개됨

3장. 강화학습 기초 2: 그리드월드와 다이내믹 프로그래밍

3.1 강화학습 알고리즘의 흐름



3.2 다이내믹 프로그래밍

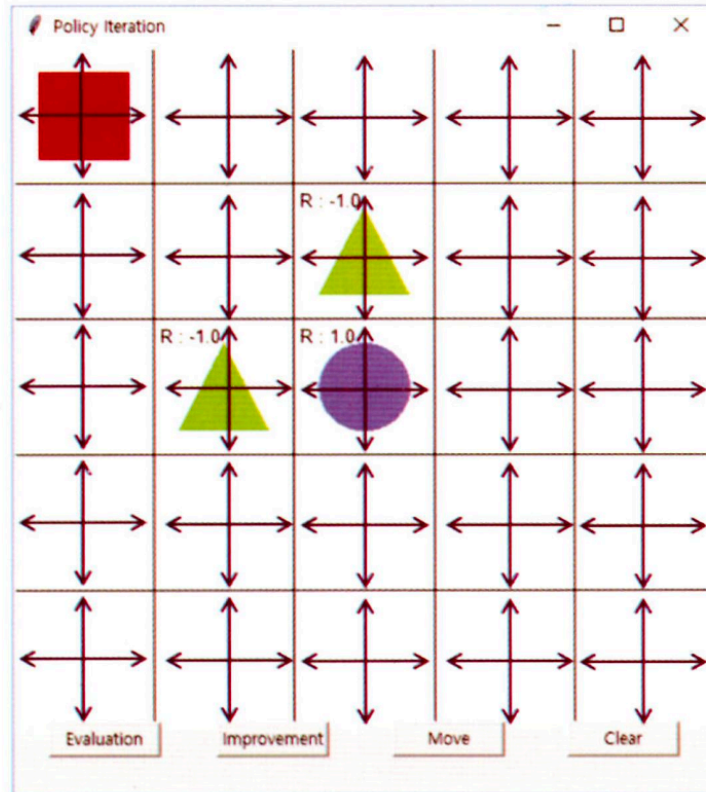
: 벨만 방정식을 반복 계산해 최적 가치함수와 최적 정책을 찾는 방법

- MDP로 정의된 문제를 푸는 기본 계산 방법
 - 상태와 행동이 명확히 정의된 환경에서 사용
 - 핵심 아이디어: 가치함수를 반복적으로 업데이트
 - 목표: 보상의 합이 최대가 되는 최적 정책 찾기
 - 대표 알고리즘
 - 정책 이터레이션
 - 가치 이터레이션
-

3.3 정책 이터레이션

: 정책 평가와 정책 발전을 번갈아 수행하며 정책을 점점 개선하는 방법

- 다이내믹 프로그래밍의 한 종류
- 벨만 기대 방정식 사용
- 처음에는 최적 정책을 모르므로 임의의 정책에서 시작
- 보통 무작위 정책으로 시작
- 이후 두 단계를 반복
 - 정책 평가
 - 정책 발전

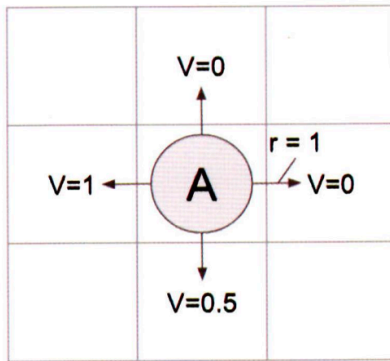


3.4 정책 평가

: 현재 정책을 따른다고 가정했을 때 각 상태의 가치함수를 계산하는 단계

- 현재 정책 고정
- 그 정책 아래에서 각 상태의 가치 계산
- 벨만 기대 방정식을 반복 적용
- 한 번 계산하고 끝나는 것이 아니라 수렴할 때까지 반복
- 의미
 - 지금 정책이 얼마나 좋은지 수치화
 - 다음 단계인 정책 발전의 기준 제공

$$v_{\pi}(s) = E_{\pi}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \cdots | S_t = s]$$



$$\text{상} : 0.25 \times (0 + 0.9 \times 0) = 0$$

$$\text{하} : 0.25 \times (0 + 0.9 \times 0.5) = 0.1125$$

$$\text{좌} : 0.25 \times (0 + 0.9 \times 1) = 0.225$$

$$\text{우} : 0.25 \times (1 + 0.9 \times 0) = 0.25$$

$$\text{다음 가치함수} = 0 + 0.1125 + 0.225 + 0.25$$

3.5 정책 발전

: 현재 가치함수를 기준으로 더 좋은 행동을 고르는 새 정책으로 업데이트하는 단계

- 정책 평가로 얻은 가치함수를 바탕으로 행동 선택
- 각 상태에서 가능한 행동들의 큐함수 비교
- 가장 큰 값을 주는 행동 선택
- **탐욕 정책 발전 greedy policy improvement**
- 탐욕 정책 발전 후의 정책은 이전 정책보다 가치가 크거나 같음
- 따라서 반복하면 최적 정책으로 수렴 가능

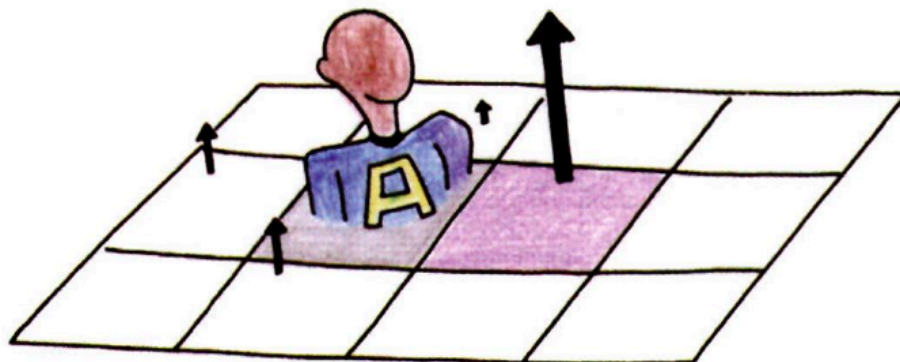


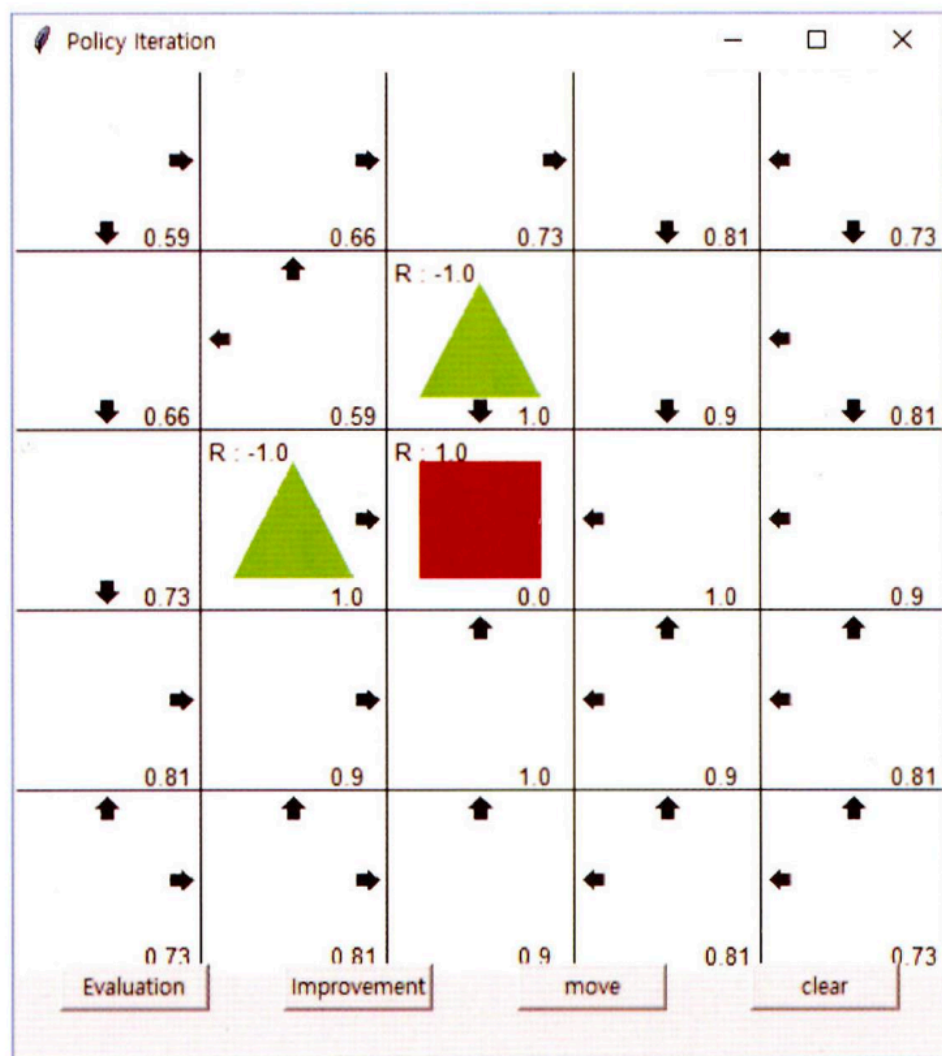
그림 3.9 큐함수의 값이 제일 높은 행동을 선택하는 탐욕 정책 발전

3.6 정책 이터레이션의 반복 구조

: 정책 평가 → 정책 발전을 반복하면 정책과 가치함수가 함께 개선됨

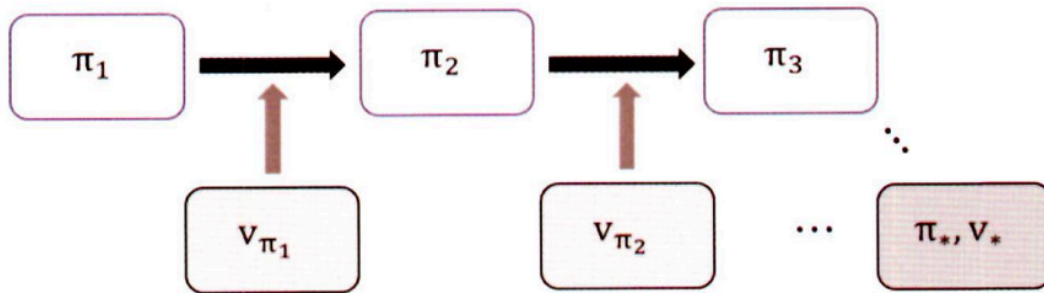
- 현재 정책으로 가치함수 계산

- 계산된 가치함수로 더 좋은 정책 생성
- 새 정책으로 다시 가치함수 계산
- 이 과정을 반복
- 결과
 - 정책이 점점 명확해짐
 - 가치함수도 점점 커지고 안정화
 - 최적 정책과 최적 가치함수에 수렴



3.7 가치 이터레이션

: 정책을 따로 분리하지 않고 벨만 최적 방정식을 직접 반복 계산하는 방법



- 정책 이터레이션은 정책 평가를 수렴할 때까지 반복해야 해서 느릴 수 있음
- 가치 이터레이션은 정책을 명시적으로 두지 않음
- 가치함수 안에 정책이 **내재적 implicit**으로 포함된다고 봄
- 벨만 최적 방정식을 바로 사용해 가치함수 업데이트
- 매 스텝마다 최댓값을 취함
- 결과적으로 더 빠르게 최적 가치함수에 접근 가능

$$v_{k+1}(s) = \max_{a \in A} (r_{(s,a)} + \gamma v_k(s'))$$

- 정책 이터레이션: 정책과 가치함수 분리
- 가치 이터레이션: 가치함수만 직접 업데이트

```

import numpy as np
from environment import GraphicDisplay, Env

class ValueIteration:
    def __init__(self, env):
        # 환경에 대한 객체 선언
        self.env = env
        # 가치함수를 2차원 리스트로 초기화
        self.value_table = [[0.0] * env.width for _ in range(env.height)]
        # 할인율
        self.discount_factor = 0.9

    # 벨만 최적 방정식을 통해 다음 가치함수 계산
    def value_iteration(self):
        # 다음 가치함수 초기화
        next_value_table = [[0.0] * self.env.width
                             for _ in range(self.env.height)]

        # 모든 상태에 대해서 벨만 최적방정식을 계산
        for state in self.env.get_all_states():
            # 마침 상태의 가치함수 = 0
            if state == [2, 2]:
                next_value_table[state[0]][state[1]] = 0.0

```